

使用 KNN 算法预测第二天气温（回归问题）



完整代码实现

代码块

```
1  import pandas as pd
2  import numpy as np
3  from sklearn.neighbors import KNeighborsRegressor
4  from sklearn.model_selection import train_test_split, cross_val_score,
   GridSearchCV
5  from sklearn.preprocessing import StandardScaler
6  from sklearn.metrics import mean_squared_error, mean_absolute_error, r2_score
7  import matplotlib.pyplot as plt
8  import warnings
9  warnings.filterwarnings('ignore')
10
11  # ===== 1. 加载数据 =====
12  print("="*50)
13  print("1. 加载数据")
14  print("="*50)
15
16  df = pd.read_csv('data/weather_data.csv')
17  print(f"\n数据形状: {df.shape}")
18  print(f"\n数据预览: ")
19  print(df)
20  print(f"\n数据统计信息: ")
21  print(df.describe())
22
23  # ===== 2. 数据预处理 =====
24  print("\n" + "="*50)
25  print("2. 数据预处理")
26  print("="*50)
27
28  # 分离特征和目标变量
29  X = df[['temperature', 'humidity', 'wind_speed', 'rain_probability']]
30  y = df['y']
31
32  print(f"\n特征矩阵 X 形状: {X.shape}")
33  print(f"\n目标变量 y 形状: {y.shape}")
34
35  # 划分训练集和测试集（由于数据量小，使用较大比例训练）
```

```

36 X_train, X_test, y_train, y_test = train_test_split(
37     X, y, test_size=0.2, random_state=42
38 )
39
40 print(f"\n训练集大小: {X_train.shape[0]}")
41 print(f"\n测试集大小: {X_test.shape[0]}")
42
43 # 数据标准化 (KNN 对特征尺度敏感, 必须标准化)
44 scaler = StandardScaler()
45 X_train_scaled = scaler.fit_transform(X_train)
46 X_test_scaled = scaler.transform(X_test)
47
48 print("\n标准化完成! ")
49
50 # ===== 3. 训练 KNN 回归模型 =====
51 print("\n" + "="*50)
52 print("3. 训练 KNN 回归模型")
53 print("="*50)
54
55 # 创建 KNN 回归器
56 knn_regressor = KNeighborsRegressor(
57     n_neighbors=3,          # K 值
58     weights='distance',    # 距离加权 (距离近的邻居权重更大)
59     metric='euclidean',    # 欧氏距离
60     p=2                    # 闵可夫斯基距离参数
61 )
62
63 # 训练模型
64 knn_regressor.fit(X_train_scaled, y_train)
65 print("\n模型训练完成! ")
66
67 # ===== 4. 预测与评估 =====
68 print("\n" + "="*50)
69 print("4. 模型预测与评估")
70 print("="*50)
71
72 # 预测
73 y_pred = knn_regressor.predict(X_test_scaled)
74
75 # 计算评估指标
76 mse = mean_squared_error(y_test, y_pred)
77 rmse = np.sqrt(mse)
78 mae = mean_absolute_error(y_test, y_pred)
79 r2 = r2_score(y_test, y_pred)
80
81 print(f"\n 模型评估结果: ")
82 print(f"{'='*40}")

```

```

83 print(f"均方误差 (MSE) : {mse:.4f}")
84 print(f"均方根误差 (RMSE): {rmse:.4f}")
85 print(f"平均绝对误差 (MAE): {mae:.4f}")
86 print(f"决定系数 (R²) : {r2:.4f}")
87 print(f"{'='*40}")
88
89 # 显示预测结果对比
90 print(f"\n📊 预测结果对比: ")
91 print(f"{'='*40}")
92 print(f"{'真实值':<10} {'预测值':<10} {'误差':<10}")
93 print(f"{'-'*40}")
94 for true, pred in zip(y_test.values, y_pred):
95     error = abs(true - pred)
96     print(f"{true:<12.2f} {pred:<12.2f} {error:<12.2f}")
97
98 # ===== 5. K 值优化 (交叉验证) =====
99 print("\n" + "="*50)
100 print("5. K 值优化 (交叉验证) ")
101 print("="*50)
102
103 # 测试不同的 K 值
104 k_range = range(1, min(11, len(X_train))) # K 值范围 1-10
105 cv_scores = []
106
107 for k in k_range:
108     knn = KNeighborsRegressor(n_neighbors=k, weights='distance')
109     scores = cross_val_score(knn, X_train_scaled, y_train, cv=3,
110 scoring='neg_mean_squared_error')
111     cv_scores.append(-scores.mean()) # 转换为正数
112
113 # 找到最佳 K 值
114 best_k = k_range[np.argmin(cv_scores)]
115 print(f"\n不同 K 值的 MSE: ")
116 print(f"{'K 值':<10} {'MSE':<10}")
117 print(f"{'-'*20}")
118 for k, score in zip(k_range, cv_scores):
119     print(f"{k:<12} {score:.4f}")
120
121 print(f"\n🌟 最佳 K 值: {best_k}")
122
123 # 使用最佳 K 值重新训练
124 best_knn = KNeighborsRegressor(n_neighbors=best_k, weights='distance')
125 best_knn.fit(X_train_scaled, y_train)
126 y_pred_best = best_knn.predict(X_test_scaled)
127
128 print(f"\n使用最佳 K 值 ({best_k}) 的模型评估: ")
129 print(f"MSE : {mean_squared_error(y_test, y_pred_best):.4f}")

```

```

129 print(f"RMSE: {np.sqrt(mean_squared_error(y_test, y_pred_best)):.4f}")
130 print(f"R² : {r2_score(y_test, y_pred_best):.4f}")
131
132 # ===== 6. 可视化 =====
133 print("\n" + "="*50)
134 print("6. 可视化结果")
135 print("="*50)
136
137 # 创建图表
138 fig, axes = plt.subplots(1, 2, figsize=(12, 5))
139
140 # 图 1: K 值与 MSE 的关系
141 axes[0].plot(k_range, cv_scores, marker='o', linewidth=2, markersize=8)
142 axes[0].set_xlabel('K Value', fontsize=12)
143 axes[0].set_ylabel('MSE', fontsize=12)
144 axes[0].set_title('K Value vs MSE', fontsize=14)
145 axes[0].grid(True, alpha=0.3)
146 axes[0].annotate(f'Best K={best_k}',
147                  xy=(best_k, cv_scores[best_k-1]),
148                  xytext=(best_k+1, cv_scores[best_k-1]+5),
149                  arrowprops=dict(arrowstyle='->', color='red'),
150                  fontsize=10, color='red')
151
152 # 图 2: 真实值 vs 预测值
153 axes[1].scatter(y_test.values, y_pred_best, s=100, color='blue', alpha=0.7)
154 axes[1].plot([y_test.min(), y_test.max()], [y_test.min(), y_test.max()],
155              'r--', lw=2, label='Perfect Prediction')
156 axes[1].set_xlabel('True Values', fontsize=12)
157 axes[1].set_ylabel('Predicted Values', fontsize=12)
158 axes[1].set_title('True vs Predicted', fontsize=14)
159 axes[1].legend()
160 axes[1].grid(True, alpha=0.3)
161
162 plt.tight_layout()
163 plt.show()
164
165 # ===== 7. 新数据预测示例 =====
166 print("\n" + "="*50)
167 print("7. 新数据预测示例")
168 print("="*50)
169
170 # 创建新的天气数据
171 new_weather_data = pd.DataFrame({
172     'temperature': [26],
173     'humidity': [75],
174     'wind_speed': [10],
175     'rain_probability': [20]

```

```

176     })
177
178     print(f"\n 今天的天气数据: ")
179     print(new_weather_data)
180
181     # 标准化新数据
182     new_weather_scaled = scaler.transform(new_weather_data)
183
184     # 预测明天的温度
185     predicted_temp = best_knn.predict(new_weather_scaled)
186     print(f"\n🌤️ 预测明天的温度: {predicted_temp[0]:.2f}°C")
187
188     # 找出最相似的邻居
189     distances, indices = best_knn.kneighbors(new_weather_scaled)
190     print(f"\n📍 最相似的 {best_k} 个邻居: ")
191     print(f"{'索引':<8} {'距离':<10} {'明天温度':<10}")
192     print(f"{'-'*28}")
193     for i, (idx, dist) in enumerate(zip(indices[0], distances[0])):
194         print(f"{'idx':<10} {'dist':<12.4f} {y.iloc[idx]:<12.2f}")
195
196     print("\n" + "="*50)
197     print("✅ 任务完成! ")
198     print("="*50)

```

代码说明

🔑 关键步骤:

1. 数据加载与探索

- 读取 CSV 文件
- 查看数据统计信息

2. 数据预处理

- 分离特征 (X) 和目标变量 (y)
- 划分训练集和测试集

- **标准化** (KNN 必须步骤)

3. 模型训练

- 使用 `KNeighborsRegressor`
- 设置 $K=3$, 距离加权

4. 模型评估

- MSE、RMSE、MAE、 R^2
- 预测结果对比

5. K 值优化

- 交叉验证选择最佳 K 值
- 可视化 K 值与性能关系

6. 实际应用

- 新数据预测示例
- 显示最相似的邻居

重要提示：

代码块

- ```
1 # 由于数据量很小（只有 10 条），建议：
2 # 1. 使用交叉验证而不是简单的训练/测试划分
3 # 2. K 值不要太大（1-5 比较合适）
4 # 3. 结果仅供参考，实际应用中需要更多数据
```

## 核心要点：

- **回归 vs 分类**: 这里使用 `KNeighborsRegressor` 而不是 `KNeighborsClassifier`
- **标准化**: 必须对特征进行标准化
- **K 值选择**: 小数据集用较小的 K 值
- **距离加权**: `weights='distance'` 让近的邻居影响更大

运行这段代码即可得到完整的 KNN 回归预测结果! 🌡️ ✨